

eda_gym_exercice_dataset_v1.3

November 6, 2025

1 Gym_Exercise_Dataset.csv

1.1 Chargement et inspection initiale

1.1.1 Objectif

Vérifier que le fichier `Gym_Exercise_Dataset.csv` est correctement chargé et obtenir une vue d'ensemble : - dimensions du jeu de données (nombre de lignes et de colonnes), - typage des variables (numériques, catégorielles, textuelles), - détection des valeurs manquantes et doublons, - cohérence générale du corpus avant l'analyse statistique.

Cette étape permet de valider la structure et la qualité du jeu de données avant toute exploration descriptive ou visuelle.

1.1.2 Interprétation attendue

- Le dataset doit contenir une liste d'exercices accompagnés de leurs descriptions (`Title`, `Desc`) et de métadonnées associées (`Type`, `BodyPart`, `Equipment`, `Level`, `Rating`, `RatingDesc`).
- Les types doivent être correctement identifiés : **textuels** pour les descriptions et catégories, **numériques** pour les éventuelles notes.
- L'absence de valeurs manquantes et de doublons confirmera la fiabilité du corpus.
- Les noms de colonnes doivent être homogènes et cohérents pour permettre les traitements automatiques à venir.

1.1.3 Importation des librairies essentielles

```
[26]: import os
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.colors import LogNorm
import seaborn as sns
import kagglehub

from scipy import stats
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

# === Localiser automatiquement le package 'themes' en remontant l'arborescence
↪===
```

```

from pathlib import Path
import sys

root = Path.cwd()          # ex: .../DuckDB/eda/life_style_data
themes_root = None

for p in [root, *root.parents]:  # remonte: life_style_data -> eda -> DuckDB
    ↪-> ...
    if (p / "themes").is_dir():
        themes_root = p
        break

if themes_root is None:
    raise FileNotFoundError(
        f"Impossible de trouver le dossier 'themes' en partant de {root}. "
        "Vérifie l'arborescence (il doit exister un dossier 'themes/' quelque_
    ↪part au-dessus). "
    )

if str(themes_root) not in sys.path:
    sys.path.insert(0, str(themes_root))

print(" Ajouté au PYTHONPATH :", themes_root)
print(" Contenu de", themes_root, ":", [x.name for x in (themes_root).
    ↪iterdir()])

from themes.train_me import set_trainme_theme

```

Ajouté au PYTHONPATH : c:\Users\fbac\Desktop\Projets\Dev\Projet Alyra\DuckDB
 Contenu de c:\Users\fbac\Desktop\Projets\Dev\Projet Alyra\DuckDB : ['.conda',
 '.vscode', '.~lock.Exploration des données - v1.1.odt#', 'datasets',
 'duckdb.exe', 'eda', 'Exploration des données - v1.0.odt', 'Exploration des
 données - v1.0.pdf', 'Exploration des données - v1.1.odt', 'themes']

1.1.4 Configuration d’affichage pour plus de lisibilité

```

[3]: pd.set_option('display.max_columns', None)
     pd.set_option('display.width', 120)

```

1.1.5 Thème “TrAIIn.me”

Il applique un fond dark propre, une grille discrète, des ticks lisibles, et une palette bleus/cians cohérente.

```

[4]: # =====
     # Thème TrAIIn.me (Seaborn/Matplotlib)
     # =====

```

```
# Booléen de contrôle : True = thème TrAI.n.me (dark), False = style clair
↪ "ticks"
USE_DARK_THEME = True # modifie ici selon le contexte

# Activer le thème au début du notebook :
palette_trainme = set_trainme_theme(context="talk", font_scale=1.05,
↪ use_dark=True)
palette_trainme
```

```
[4]: ['#00E5FF', '#00BFFF', '#0A84FF', '#1E90FF', '#00FFFF', '#64D8FF']
```

1.1.6 Chargement du dataset

```
[5]: # Téléchargement du dataset depuis Kaggle
dataset_path = kagglehub.dataset_download("niharika41298/gym-exercise-data")

# Recherche du fichier CSV dans le dossier téléchargé
csv_files = [f for f in os.listdir(dataset_path) if f.endswith(".csv")]
print("Fichiers trouvés :", csv_files)

# Construction du chemin complet vers le CSV
file_path = os.path.join(dataset_path, csv_files[0])

# Lecture du fichier CSV
df = pd.read_csv(file_path)

# Informations de confirmation
print("Dataset chargé avec succès !")
print(f"Dimensions : {df.shape[0]} lignes et {df.shape[1]} colonnes\n")
```

Warning: Looks like you're using an outdated `kagglehub` version, please consider updating (latest version: 0.3.13)

Fichiers trouvés : ['megaGymDataset.csv']

Dataset chargé avec succès !

Dimensions : 2918 lignes et 9 colonnes

Fichiers trouvés : ['megaGymDataset.csv']

Dataset chargé avec succès !

Dimensions : 2918 lignes et 9 colonnes

1.1.7 Aperçu général des premières lignes

```
[6]: display(df.head())
```

Unnamed: 0	Title
↪ Desc	Type BodyPart \

```

0          0          Partner plank band row  The partner plank band row is an
↳abdominal exe... Strength Abdominals
1          1  Banded crunch isometric hold  The banded crunch isometric hold is
↳an exercis... Strength Abdominals
2          2          FYR Banded Plank Jack  The banded plank jack is a
↳variation on the pl... Strength Abdominals
3          3          Banded crunch  The banded crunch is an exercise
↳targeting the... Strength Abdominals
4          4          Crunch  The crunch is a popular core
↳exercise targetin... Strength Abdominals

```

	Equipment	Level	Rating	RatingDesc
0	Bands	Intermediate	0.0	NaN
1	Bands	Intermediate	NaN	NaN
2	Bands	Intermediate	NaN	NaN
3	Bands	Intermediate	NaN	NaN
4	Bands	Intermediate	NaN	NaN

1.1.8 Informations générales sur le typage et la complétude

```
[7]: df_info = df.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2918 entries, 0 to 2917
Data columns (total 9 columns):
#   Column          Non-Null Count  Dtype
---  -
0   Unnamed: 0      2918 non-null  int64
1   Title           2918 non-null  object
2   Desc            1368 non-null  object
3   Type            2918 non-null  object
4   BodyPart        2918 non-null  object
5   Equipment       2886 non-null  object
6   Level           2918 non-null  object
7   Rating          1031 non-null  float64
8   RatingDesc      862 non-null   object
dtypes: float64(1), int64(1), object(7)
memory usage: 205.3+ KB

```

1.1.9 Statistiques descriptives des variables numériques

```
[8]: display(df.describe().T)
```

	count	mean	std	min	25%	50%	75%	max
Unnamed: 0	2918.0	1458.50000	842.498368	0.0	729.25	1458.5	2187.75	2917.0
Rating	1031.0	5.91969	3.584607	0.0	3.00	7.9	8.70	9.6

1.1.10 Analyse des valeurs manquantes et doublons

```
[9]: missing_values = df.isnull().sum().sort_values(ascending=False)
duplicates = df.duplicated().sum()

print("\n--- Valeurs manquantes par colonne (top 10) ---")
display(missing_values.head(10))

print(f"\nNombre total de doublons détectés : {duplicates}")
```

--- Valeurs manquantes par colonne (top 10) ---

```
RatingDesc    2056
Rating        1887
Desc          1550
Equipment      32
Unnamed: 0      0
Title          0
Type           0
BodyPart       0
Level          0
dtype: int64
```

Nombre total de doublons détectés : 0

1.1.11 Liste complète des colonnes disponibles

```
[10]: print("\n--- Liste des colonnes du dataset ---")
print(df.columns.tolist())
```

--- Liste des colonnes du dataset ---

```
['Unnamed: 0', 'Title', 'Desc', 'Type', 'BodyPart', 'Equipment', 'Level',
'Rating', 'RatingDesc']
```

1.2 Statistiques descriptives et distribution des variables

1.2.1 Objectif

Explorer la distribution des variables numériques afin de comprendre la variabilité du dataset, identifier d'éventuelles anomalies et détecter les premiers signaux statistiques utiles.

1.2.2 Interprétation attendue

- Confirmer la cohérence statistique des variables numériques disponibles (souvent **Rating**).
- Vérifier la normalité ou l'asymétrie de la distribution (pics, étalement, queue longue).
- Repérer visuellement d'éventuels **outliers** (notes aberrantes, valeurs hors bornes métier).
- Si une seule variable numérique est présente (ex. **Rating**), ces graphiques servent surtout à juger de la dispersion et de la couverture (taux de valeurs manquantes).

1.2.3 Activation du thème TrAIn.me

```
[11]: if USE_DARK_THEME:
    palette_trainme = set_trainme_theme(context="talk", font_scale=1.05,
    ↪use_dark=True)
else:
    sns.set_style("ticks")           # style clair sobre, axes renforcés
    sns.set_context("talk", font_scale=1.05)
    palette_trainme = sns.color_palette("deep") # palette par défaut sobre
    print(" Style activé : 'ticks' → sobre, axes renforcés (présentation pro)")

# Exemple de vérification :
print("Palette active :", palette_trainme.as_hex() if hasattr(palette_trainme,
    ↪"as_hex") else palette_trainme)
```

Palette active : ['#00E5FF', '#00BFFF', '#0A84FF', '#1E90FF', '#00FFFF',
'#64D8FF']

1.2.4 Statistiques descriptives globales

```
[13]: # On limite aux colonnes numériques utiles (exclut un éventuel index 'Unnamed:
    ↪0')
num_df = df.select_dtypes(include=[np.number]).copy()
for col in ["Unnamed: 0", "index", "Id", "ID"]:
    if col in num_df.columns:
        num_df = num_df.drop(columns=[col])

stats = num_df.describe().T
display(stats if not stats.empty else pd.DataFrame({"info": ["Aucune variable
    ↪numérique exploitable"]}))
```

	count	mean	std	min	25%	50%	75%	max
Rating	1031.0	5.91969	3.584607	0.0	3.0	7.9	8.7	9.6

1.2.5 Distribution des variables clés

```
[14]: # Si liste vide → on tente d'utiliser 'Rating' si présent (souvent la seule
    ↪numérique pertinente)
num_vars = list(num_df.columns)
if not num_vars and "Rating" in df.columns:
    num_vars = ["Rating"]

if num_vars:
    n = len(num_vars)
    ncols = 3
    nrows = (n + ncols - 1) // ncols

    plt.figure(figsize=(4*ncols, 3*nrows + 1))
```

```

for i, col in enumerate(num_vars, 1):
    plt.subplot(nrows, ncols, i)
    sns.histplot(df[col].dropna(), bins=30, kde=True,
    color=palette_trainme[i % len(palette_trainme)])
    plt.title(col, fontsize=11, pad=8)
    plt.xlabel(col)
    plt.ylabel("Fréquence")
    plt.tight_layout()
    plt.show()
else:
    print(" Aucune variable numérique à tracer (vérifie la présence de
    'Rating').")

```



1.2.6 Boxplots pour repérer les outliers

```

[15]: if num_vars:
    n = len(num_vars)
    ncols = 3
    nrows = (n + ncols - 1) // ncols

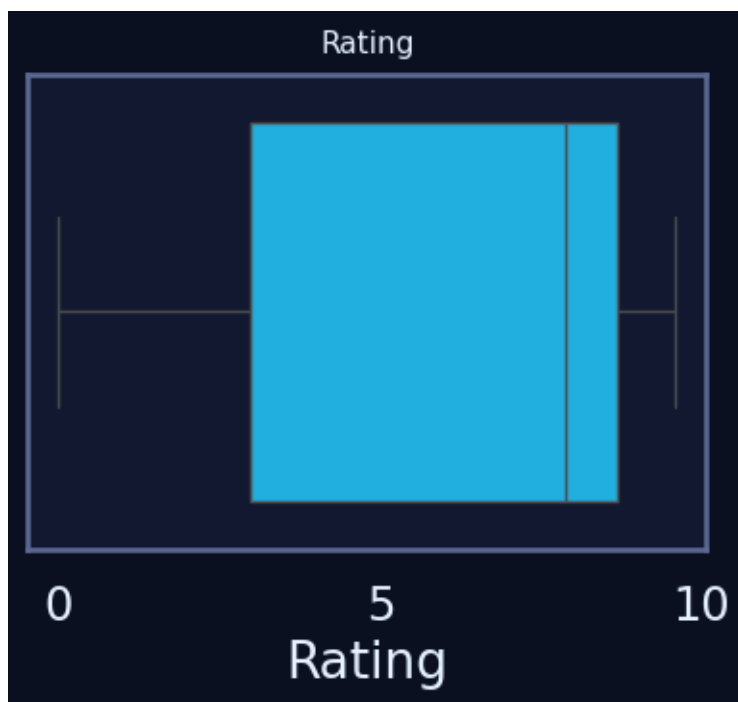
    plt.figure(figsize=(4*ncols, 3*nrows + 1))
    for i, col in enumerate(num_vars, 1):
        plt.subplot(nrows, ncols, i)
        sns.boxplot(x=df[col], color=palette_trainme[i % len(palette_trainme)])

```

```

plt.title(col, fontsize=11, pad=8)
plt.xlabel(col)
plt.tight_layout()
plt.show()
else:
    print(" Pas de boxplots affichés (aucune variable numérique).")

```



1.2.7 Statistiques descriptives enrichies

```

[16]: # Sélection des variables numériques pertinentes
num_cols = df.select_dtypes(include="number").columns.tolist()

# Calcul des statistiques enrichies
desc = df[num_cols].describe().T
desc["missing_%"] = df[num_cols].isna().mean() * 100
desc["zeros_%"] = (df[num_cols] == 0).mean() * 100
desc["skew"] = df[num_cols].skew() # asymétrie
desc["kurt"] = df[num_cols].kurt() # aplatissement

# Affichage des 12 variables les plus variables
display(desc.sort_values("std", ascending=False).head(12))

```

	count	mean	std	min	25%	50%	75%	max
missing_%								
zeros_%								
skew								

```

Unnamed: 0    2918.0   1458.50000   842.498368   0.0   729.25   1458.5   2187.75   2917.0
↳ 0.000000   0.034270   0.000000
Rating        1031.0     5.91969   3.584607   0.0     3.00     7.9     8.70     9.6
↳ 64.667581   8.361892  -0.849343

```

```

                kurt
Unnamed: 0    -1.200000
Rating        -1.009552

```

1.2.8 Détection rapide d'outliers (IQR)

```

[17]: def iqr_outlier_ratio(s: pd.Series, k: float = 1.5) -> float:
        """Renvoie le pourcentage d'outliers selon la règle IQR."""
        q1, q3 = s.quantile(0.25), s.quantile(0.75)
        iqr = q3 - q1
        if iqr == 0:
            return 0.0
        low, high = q1 - k * iqr, q3 + k * iqr
        return ((s < low) | (s > high)).mean() * 100

# Calcul du taux d'outliers pour chaque variable numérique
outlier_report = pd.Series({c: iqr_outlier_ratio(df[c]) for c in num_cols},
↳ name="outliers_%")

# Affichage des 12 variables les plus affectées
display(outlier_report.sort_values(ascending=False).head(12))

```

```

Unnamed: 0    0.0
Rating        0.0
Name: outliers_%, dtype: float64

```

1.2.9 Aperçu corrélations (sous-ensemble lisible)

```

[19]: # subset_for_corr = [
#       c for c in [
#           "Calories_Burned", "BMI", "Age", "Max_BPM", "Avg_BPM",
#           "Session_Duration (hours)", "expected_burn", "cal_balance"
#       ] if c in num_cols
#     ]

# corr = df[subset_for_corr].corr(numeric_only=True)

# plt.figure(figsize=(7, 5))
# sns.heatmap(
#     corr,
#     annot=True, fmt=".2f",
#     square=True, cbar=True,
#     annot_kws={"size": 8} # taille de police ajustée

```

```
# )
# plt.title("Corrélations (sous-ensemble)", fontsize=12)
# plt.xticks(fontsize=9, rotation=45, ha="right")
# plt.yticks(fontsize=9)
# plt.tight_layout()
# plt.show()
```

1.3 Visualisation univariée

1.3.1 Objectif

Analyser la distribution individuelle des principales variables **numériques** pour : - détecter les asymétries ou concentrations atypiques, - identifier les profils dominants du dataset, - valider la cohérence des mesures (ex. notes/ratings).

1.3.2 Interprétation attendue

- Les variables numériques disponibles (souvent **Rating**) doivent présenter des distributions cohérentes (plage logique, absence de valeurs aberrantes).
- Une **asymétrie** marquée ou des **pics** inhabituels suggèrent des sous-populations (exercices massivement appréciés ou très peu notés).
- Les **boxplots** aident à repérer d'éventuels **outliers** : ils devront être contrôlés (erreurs, cas réels rares) avant la modélisation.
- Si une seule numérique est présente, la lecture se concentre sur sa **dispersion** et sa **couverture** (taux de valeurs manquantes).

1.3.3 Sélection robuste des variables numériques à visualiser

```
[20]: # On part des colonnes numériques existantes et on exclut les index/id
num_df = df.select_dtypes(include=[np.number]).copy()
for col in ["Unnamed: 0", "index", "Id", "ID"]:
    if col in num_df.columns:
        num_df.drop(columns=[col], inplace=True)

# Variables clés à visualiser (si dispo) : priorité à Rating
preferred_order = ["Rating"] # ajoute ici d'autres colonnes si elles existent_
    ↳ côté Gym
uni_vars = [c for c in preferred_order if c in num_df.columns]

# Complète avec d'autres numériques si disponibles
uni_vars += [c for c in num_df.columns if c not in uni_vars]

# Par sécurité : on limite à 9 variables pour un layout 3x3 propre
uni_vars = uni_vars[:9]

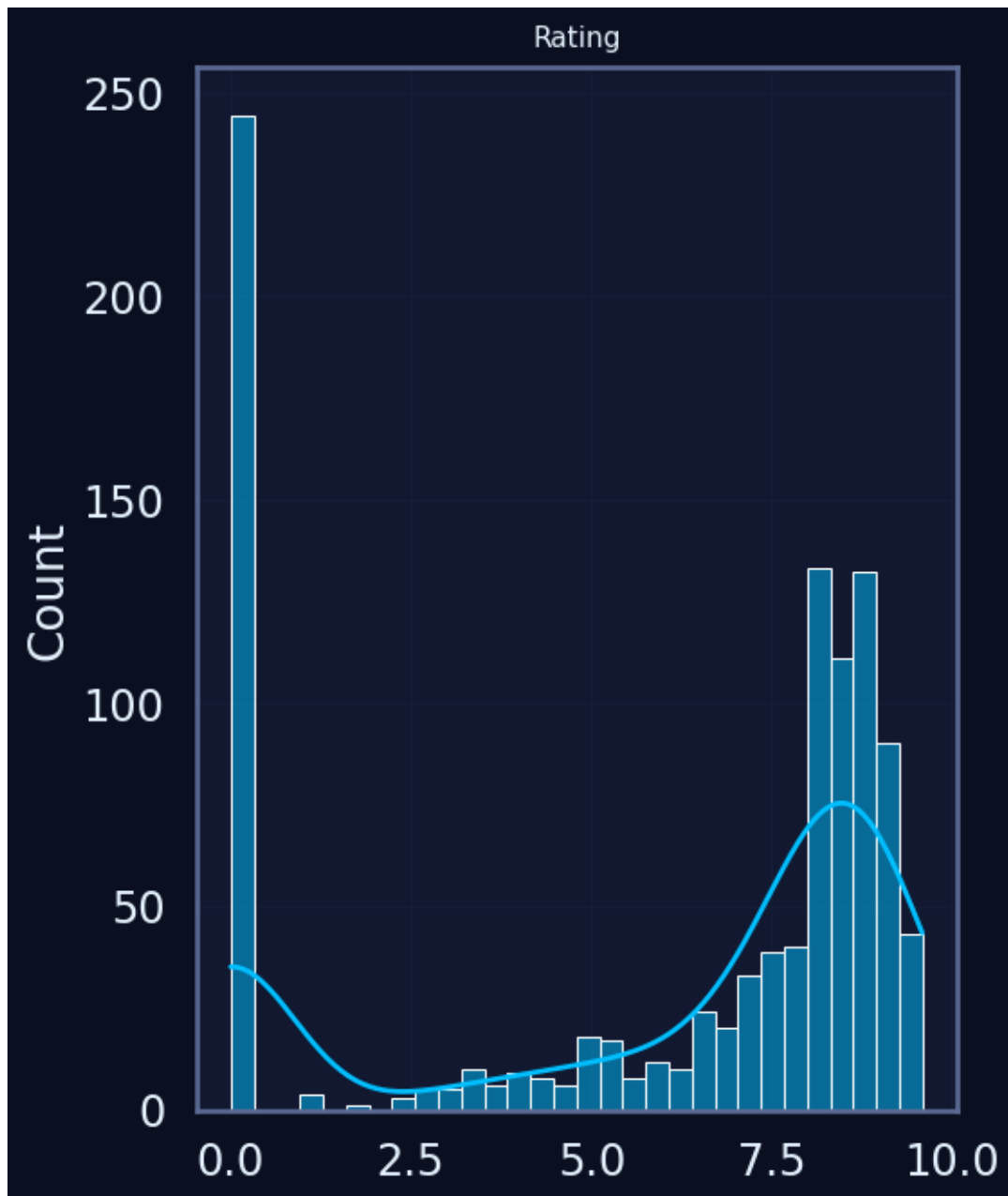
print("Variables univariées sélectionnées :", uni_vars if uni_vars else "-")
```

Variables univariées sélectionnées : ['Rating']

1.3.4 Visualisation des distributions univariées (hist + KDE)

```
[21]: if uni_vars:
    n = len(uni_vars)
    ncols = 3
    nrows = (n + ncols - 1) // ncols

    plt.figure(figsize=(15, 10 if n > 6 else 7))
    for i, col in enumerate(uni_vars, 1):
        ax = plt.subplot(nrows, ncols, i)
        sns.histplot(df[col].dropna(), kde=True,
                     color=palette_trainme[i % len(palette_trainme)], bins=30)
        ax.set_title(col, fontsize=11, pad=8)
        ax.set_xlabel("")
        ax.grid(True, alpha=0.2)
    plt.tight_layout()
    plt.show()
else:
    print(" Aucune variable numérique à tracer (vérifie la présence de_
    ↪ 'Rating').")
```



1.3.5 Boxplots univariés pour détection visuelle d'outliers

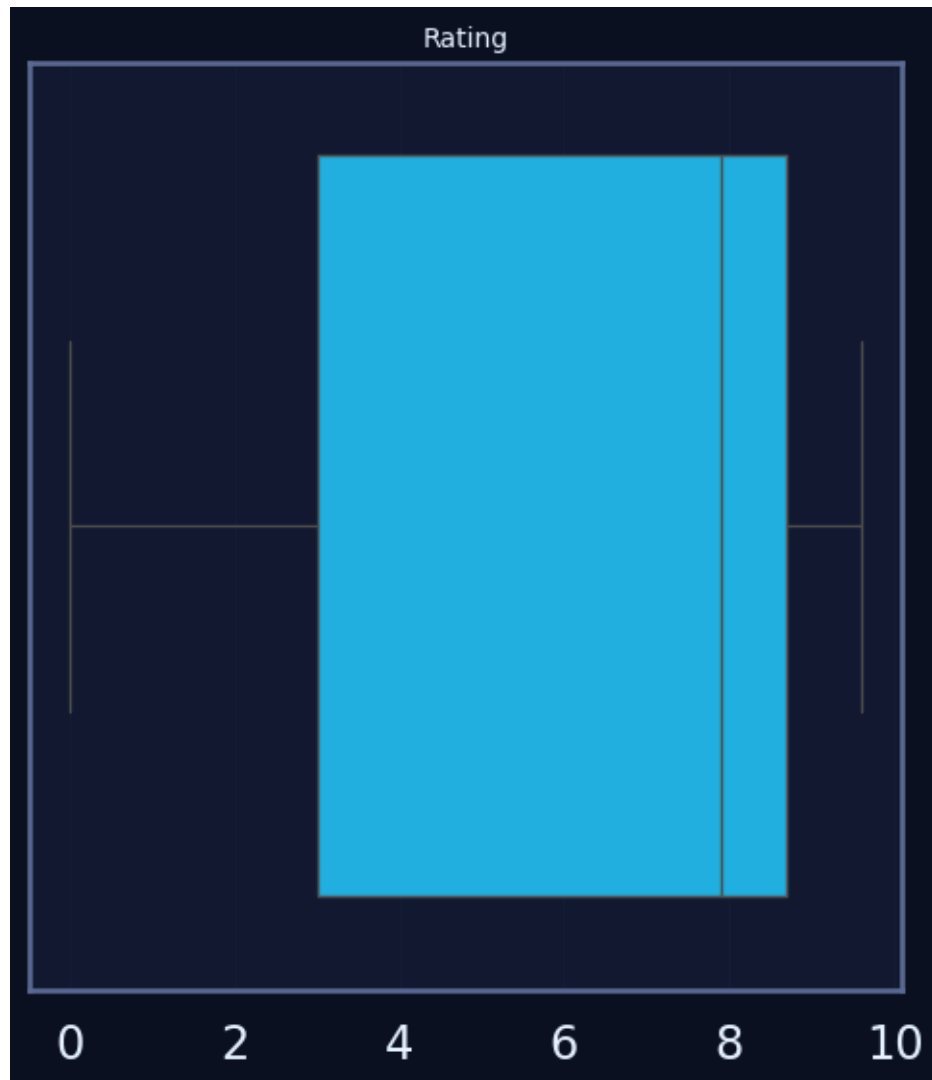
```
[22]: if uni_vars:
    n = len(uni_vars)
    ncols = 3
    nrow = (n + ncols - 1) // ncols

    plt.figure(figsize=(15, 8 if n > 6 else 6))
```

```

for i, col in enumerate(uni_vars, 1):
    ax = plt.subplot(nrows, ncols, i)
    sns.boxplot(x=df[col], color=palette_trainme[i % len(palette_trainme)],
    fliersize=2)
    ax.set_title(col, fontsize=10)
    ax.set_xlabel("")
    ax.grid(True, axis="x", alpha=0.2)
plt.tight_layout()
plt.show()
else:
    print(" Pas de boxplots affichés (aucune variable numérique).")

```



1.4 Visualisation bivariée

1.4.1 Objectif

Étudier les relations entre deux variables à la fois pour identifier : - des différences de distribution de **Rating** selon les catégories (Type, BodyPart, Equipment, Level), - des patterns d'usage (ex. Type BodyPart), - des co-occurrences fortes entre attributs décrivant les exercices.

1.4.2 Interprétation attendue

- **Catégories Rating** : si des écarts nets apparaissent entre niveaux (Level) ou familles d'exercices (Type), ils indiquent des préférences/évaluations différenciées selon la difficulté et la nature du mouvement.
- **Co-occurrences** :
 - Type × BodyPart révèle quelles zones du corps sont le plus souvent ciblées par chaque famille d'exercices.
 - Type × Equipment indique le matériel le plus mobilisé par type d'entraînement (haltères, barres, bandes, aucun).
 - Level × Type met en évidence la distribution de difficulté par type (p.ex. davantage d' "Intermediate" en "Strength").
- **Comptage empilé** : compare la mixité des *Types* à l'intérieur des équipements les plus fréquents (en nombre et en ratio), utile pour des recommandations conditionnées par le matériel disponible.

1.4.3 Relations catégorielles numérique (Rating)

```
[23]: # Vérif présence de Rating
has_rating = "Rating" in df.columns and df["Rating"].notna().any()
cat_cols = [c for c in ["Type", "BodyPart", "Equipment", "Level"] if c in df.
↳ columns]

if has_rating and cat_cols:
    # Boxplots + swarm (ou strip) pour visualiser la dispersion par catégorie
    n = len(cat_cols)
    ncols = 2
    nrows = (n + ncols - 1) // ncols

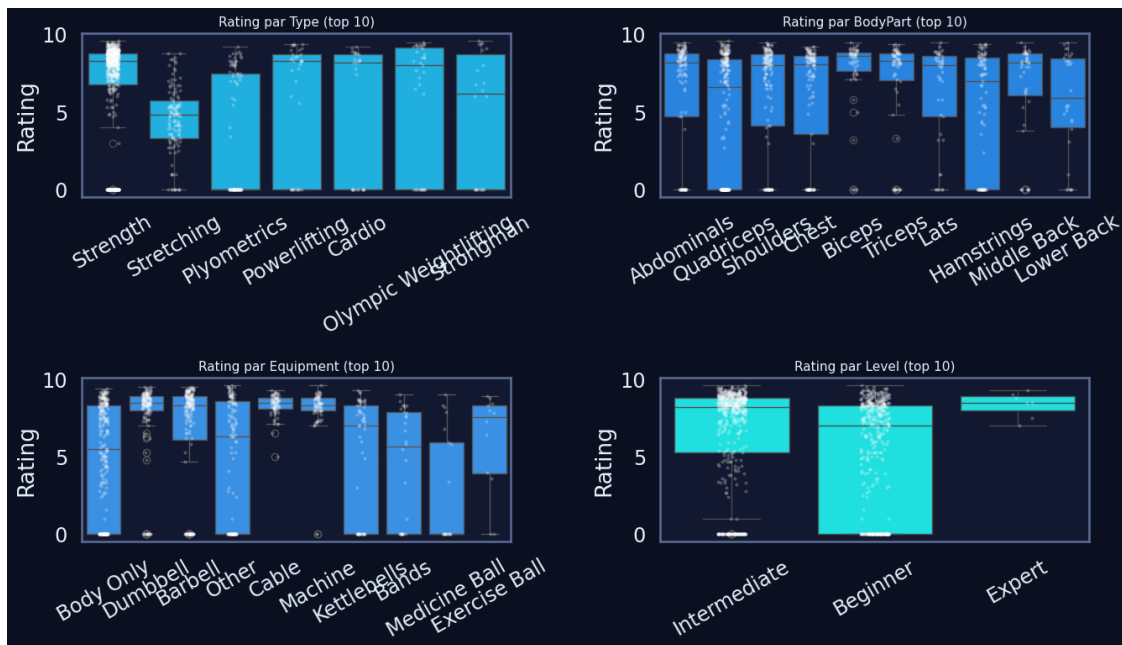
    plt.figure(figsize=(14, 4*nrows))
    for i, c in enumerate(cat_cols, 1):
        ax = plt.subplot(nrows, ncols, i)
        topk = df[c].value_counts().index[:10] # limiter à top 10 pour↳
        lisibilité
        sub = df[df[c].isin(topk)].copy()
        order = sub[c].value_counts().index

        sns.boxplot(data=sub, x=c, y="Rating", order=order,
                    color=palette_trainme[i % len(palette_trainme)], ax=ax)
        sns.stripplot(data=sub, x=c, y="Rating", order=order, ax=ax,
                    alpha=0.25, size=3, color="white")
```

```

ax.set_title(f"Rating par {c} (top 10)", fontsize=11)
ax.set_xlabel("")
ax.set_ylabel("Rating")
ax.tick_params(axis='x', rotation=30)
ax.grid(True, axis="y", alpha=0.2)
plt.tight_layout()
plt.show()
else:
    print(" Analyse 'catégorie Rating' non effectuée (colonne 'Rating' ↵
↵absente ou vide).")

```



1.4.4 Co-occurrences entre catégories (crosstab + heatmaps)

```

[27]: def plot_crosstab_heatmaps(
    df,
    pairs,
    top_k=12,
    normalize=None,      # None / "row" / "col" (pourcentages)
    annotate=False,      # True pour afficher les valeurs
    log_color=False,     # True pour mieux voir les faibles valeurs
    figsize=(16, 9),
    title_fontsize=13,
    tick_fontsize=9,
    label_rotation_x=30,
    label_rotation_y=0,
    cbar_shrink=0.85

```

```

):
    """Heatmaps propres et lisibles pour crosstabs."""
    if not pairs:
        print(" Aucune paire de catégories disponible.")
        return

    n = len(pairs)
    ncols = 2
    nrows = (n + ncols - 1) // ncols
    fig, axes = plt.subplots(nrows=nrows, ncols=ncols, figsize=figsize)
    axes = np.atleast_1d(axes).ravel()

    for i, (a, b) in enumerate(pairs):
        ax = axes[i]

        # Top-k pour réduire la cardinalité
        a_top = df[a].value_counts().index[:top_k]
        b_top = df[b].value_counts().index[:top_k]
        sub = df[df[a].isin(a_top) & df[b].isin(b_top)]

        ct = pd.crosstab(sub[a], sub[b])

        # Normalisation optionnelle en pourcentages
        if normalize in {"row", "col"}:
            axis = 1 if normalize == "row" else 0
            ct = (ct.div(ct.sum(axis=axis).replace(0, np.nan), axis=1-axis) *
↪100).round(1)

        # Palette & échelle adaptées au dark
        heatmap_kwargs = dict(
            cmap="crest",
            linewidths=0.4,
            linecolor="grey",
            cbar_kws={"shrink": cbar_shrink},
            square=False
        )

        if log_color and normalize is None:
            # Log uniquement pour les comptes bruts (pas en %)
            heatmap_kwargs["norm"] = LogNorm(vmin=max(ct.replace(0, np.nan).
↪min().min(), 1e-6),
                                                vmax=ct.max().max())

        # Annotations optionnelles (évite le bruit sur grands tableaux)
        if annotate:
            sns.heatmap(ct, ax=ax, **heatmap_kwargs, annot=True, fmt=".1f" if
↪normalize else "g",
                        annot_kws={"fontsize": max(tick_fontsize-1, 7)})

```

```

else:
    sns.heatmap(ct, ax=ax, **heatmap_kwargs)

    ax.set_title(f"Co-occurrences {a} × {b} (top {top_k})",
    ↪fontsize=title_fontsize, pad=8)
    ax.set_xlabel(b, fontsize=10)
    ax.set_ylabel(a, fontsize=10)

    # Ticks propres
    ax.tick_params(axis="x", labelsize=tick_fontsize,
    ↪rotation=label_rotation_x)
    ax.tick_params(axis="y", labelsize=tick_fontsize,
    ↪rotation=label_rotation_y)
    ax.grid(False)

    # Masquer axes vides si nombre impair
    for j in range(i+1, nrows*ncols):
        fig.delaxes(axes[j])

plt.tight_layout()
plt.show()

# =====
# Sélection des paires & appel
# =====
pairs = []
if {"Type", "BodyPart"}.issubset(df.columns):
    pairs.append(("Type",
    ↪"BodyPart"))
if {"Type", "Equipment"}.issubset(df.columns):
    pairs.append(("Type",
    ↪"Equipment"))
if {"BodyPart", "Equipment"}.issubset(df.columns):
    pairs.append(("BodyPart",
    ↪"Equipment"))
if {"Level", "Type"}.issubset(df.columns):
    pairs.append(("Level",
    ↪"Type"))

# Choix recommandés pour la lisibilité de ta capture :
# - normalize="row" pour lire des ratios par ligne (plus comparables)
# - annotate=False pour éviter la surcharge visuelle
# - log_color=False (inutile en %)
plot_crosstab_heatmaps(
    df,
    pairs=pairs,
    top_k=12,
    normalize="row",      # None pour comptes bruts ; "row" ou "col" pour %
    annotate=False,      # passe à True si tu veux les valeurs dans les cases

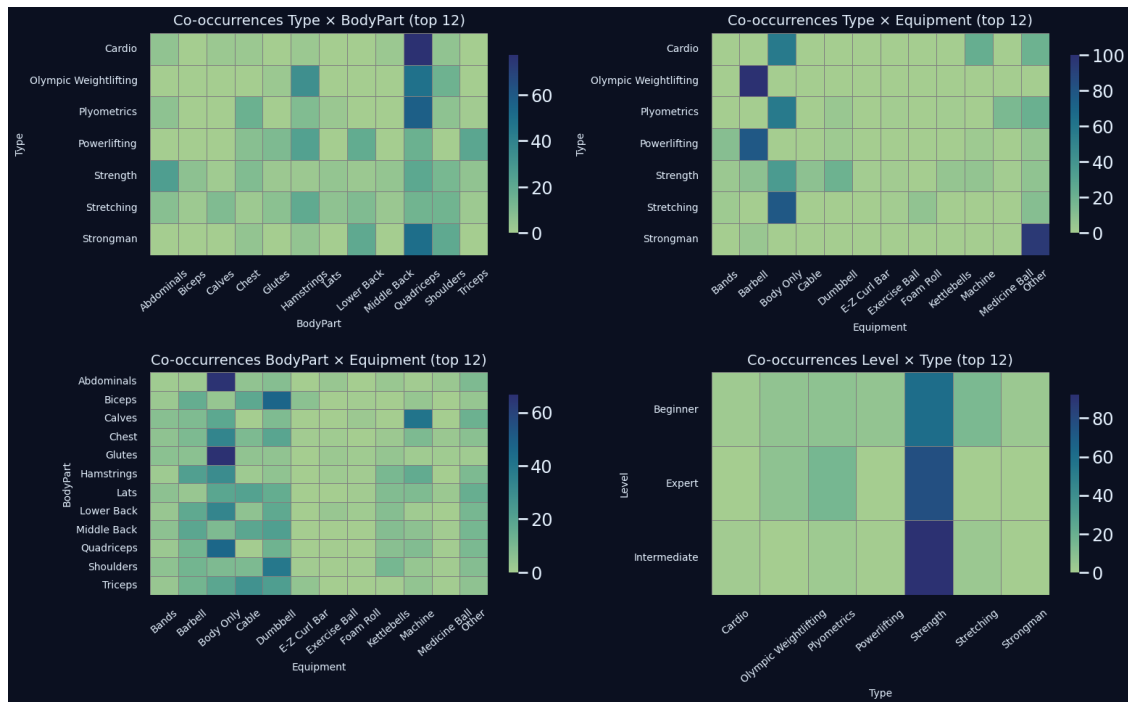
```

```

log_color=False,      # True intéressant si comptes bruts très déséquilibrés
figsize=(16, 10),
title_fontsize=14,
tick_fontsize=10,
label_rotation_x=40, # améliore la lecture en x
label_rotation_y=0,
cbar_shrink=0.8

```

)



1.4.5 Comptage empilé (stacked) : Type au sein du top Equipment

```

[25]: if {"Equipment", "Type"}.issubset(df.columns):
    top_eq = df["Equipment"].value_counts().index[:8]
    sub = df[df["Equipment"].isin(top_eq)].copy()
    tbl = (sub.groupby(["Equipment", "Type"])
           .size()
           .unstack(fill_value=0)
           .sort_index(ascending=True))
    # Normalisation pour ratios (optionnel)
    tbl_ratio = (tbl.T / tbl.T.sum()).T

    # Affichage stacked bar (comptes)
    ax = tbl.plot(kind="bar", stacked=True, figsize=(12, 6), alpha=0.9)
    ax.set_title("Répartition des Types au sein du top 8 Equipment (comptes)")

```

```

ax.set_xlabel("Equipment"); ax.set_ylabel("Nombre")
ax.grid(True, axis="y", alpha=0.2)
plt.legend(title="Type", bbox_to_anchor=(1.02, 1), loc="upper left")
plt.tight_layout()
plt.show()

# Affichage stacked bar (ratios)
ax = tbl_ratio.plot(kind="bar", stacked=True, figsize=(12, 6), alpha=0.9)
ax.set_title("Répartition des Types au sein du top 8 Equipment (ratios)")
ax.set_xlabel("Equipment"); ax.set_ylabel("Part (%)")
ax.grid(True, axis="y", alpha=0.2)
plt.legend(title="Type", bbox_to_anchor=(1.02, 1), loc="upper left")
plt.tight_layout()
plt.show()

```

